INT161 · Basic Backend Development

Mock Exam

A practice run of the practical exam: the endpoints to build, how it is scored, and how to self-test offline.

Exam Format | Setup | Endpoints and Scoring | Self-Testing | Submission | Glossary

01

Exam Format

The lecturer called it a fire drill. It is relaxed while you practise, but the real thing feels different. The point is to know steps 1, 2, 3 before it counts.

- You get an individual spec, a markdown file, and a **project template**. The spec carries your own student ID and a secret key, so the **checker** can tell whose work it is.
- The practical is a program that must work. Open the spec in preview mode and read all of it first. An **endpoint** that does not match the spec earns nothing.
- The exam is offline. You may run **npm** install once, while online. After that, switch the network off. Turning it back on counts as cheating.
- A checker runs on your own machine and prints a score, so you know where you stand before you hand in.

> **Sample case**
>
> The practice system was Mobile Phone Distributor: brands and products, one to many. On exam day the topic may differ, but the lecturer expects it to stay at two tables with one one-to-many link.

02

Setup

Step one is getting the database. You will be given a SQL file that creates the database and fills it. If you cannot run that script, you cannot take the exam.

import the database

```
$ mysql -u root -p < database/init-database.sql
```

- If the mysql command is not found, MySQL is not on your **PATH**. Either add its bin folder to PATH, or skip the command line.
- The reliable way in the IDE: create a data source, right click it, Run SQL Script, pick the file, then refresh. You should see the mobile_distributor database with its two tables.
- Create the data source before the exam, because on the day you may have no **connection** set up.
- Set the connection in src/configs/db-config.js to match your machine, with database mobile_distributor.
- Unzip the project and check that the folder is not nested twice.

Step two: put your student ID in app.js, and mount every **route** under /api/your-id.

src/app.js

```
const STUDENT_ID = 'YOUR_STUDENT_ID';
// every route lives under /api/<your id>/...
app.get(`/api/${STUDENT_ID}/student-info`, (req, res) => {
  res.status(200).json({
    success: true,
    data: { studentId: STUDENT_ID, name: 'YOUR_NAME', secretKey: 'YOUR_SECRET_KEY' },
    timestamp: new Date().toISOString(),
  });
});
```

- If the base path is wrong, the checker cannot reach your routes and every test fails.
- The checker compares your ID and secret key against its own table. A wrong value counts as wrong.

Placeholders

The code here uses YOUR_STUDENT_ID and YOUR_SECRET_KEY. Your real ID and key come from your own spec. They are personal, so never copy another student's.

03

Endpoints and Scoring

Everything you need was taught. Here is where each piece of the spec came from.

`Identity endpoint`

Class 5: a route that returns JSON.

`Brands GET`

Classes 4 and 6: repository, service, route; 404 when missing.

`Products list`

Class 8: page, limit, offset, summary DTO and metadata.

`Product detail`

Class 8: detail DTO. A join brings in the brand name.

`POST with duplicate SKU`

Class 7: duplicate entry becomes 409.

`PUT and DELETE`

Classes 4 and 6: 200 or 204, and 404 when missing.

- Identity: GET /api/id/student-info. Status 200 with success, data (studentId, name, secretKey) and a **timestamp**.

- Brands: GET /brands returns a list, GET /brands/:id returns one, or 404 if it does not exist. Each brand has id, name, country_of_origin, website, is_active and created_at, which already match the table.
- Products list: GET /products with page (default 1) and **limit**. **Summary DTO** only: id, sku, name, brandId, category, price, status.

Products list response

```
{
  "success": true,
  "data": [
    { "id": 1, "sku": "APL-IP15PM-256-NT", "name": "iPhone 15 Pro Max",
      "brandId": 1, "category": "FLAGSHIP", "price": 48900.00, "status": "AVAILABLE" }
  ],
  "pagination": {
    "total": 75, "page": 1, "limit": 10, "totalPages": 8,
    "hasNextPage": true, "hasPrevPage": false
  },
  "timestamp": "2026-10-08T15:00:00.000Z"
}
```

- Product detail: the summary fields plus brandName, ramGb, storageGb, color, stockQuantity and releaseDate. brandName lives in the other table, so you need a join. 404 if missing.
- POST /products: 201 when saved, 409 when the SKU already exists. The **request body** uses **snake_case** names, so the DTO work goes both ways.
- PUT /products/:id: 200 when updated, 404 when the id does not exist.
- DELETE /products/:id: 204 with no body, or 404 when the id does not exist.

POST /products request body

```
{
  "sku": "TEST-SKU-999", "name": "Test Phone 2026", "brand_id": 1,
  "category": "FLAGSHIP", "price": 29900.00, "ram_gb": 8, "storage_gb": 256,
  "color": "Black", "stock_quantity": 10, "status": "AVAILABLE",
  "release_date": "2026-01-01"
}
```

Tick the endpoints that pass to tally the itemized points.

☐	`GET /student-info`	**10**
☐	`GET /brands and /brands/:id`	**10**
☐	`GET /products (page, limit, summary DTO)`	**30**
☐	`GET /products/:id (detail DTO)`	**10**
☐	`POST /products`	**10**
☐	`PUT /products/:id`	**10**
☐	`DELETE /products/:id`	**5**

Itemized score: 0 of 85

Each **endpoint** is scored on its own, so a half-working project still earns points. The spec says the full total is 100, but its sections add up to 85. The other 15 are not itemized.

04

Self-Testing

Run your **server** on **port** 3000, then run the checker in a second terminal, with your own ID.

two terminals

```
# terminal 1
$ npm start
# terminal 2
$ node tests/test-runner.mjs YOUR_STUDENT_ID
```

- The checker uses the same test file for everyone, but calls the endpoints of the ID you give it.

- It prints the result of every question and the total. Each question scores on its own, so a project that does only part of the spec still earns points.
- A failing test prints a Reason, such as expected 200 and got 404. Read it, check the **status code** and the fields in the body, fix one thing, run again.
- Short on time? Start with question one. It is one small route worth 10 points.

> **Order of work**
> Do one layer at a time and test as you go, as in class 6. Then the checker only confirms what you already know.

05

Submission

Check these before you submit.

1. app.js has your own student ID.
2. The identity endpoint returns your name and your secret key.
3. The checker passes every case it can on your machine.
4. Zip only the work folder, with node_modules deleted, and name it your-id_exam.zip.

> **Practice run**
> In class the practice was not submitted for real. If you are unsure, you have two weeks to rehearse the whole thing. Go in order: import the database, set the ID, build, run the checker, zip.

06

Glossary

body
: The data a request carries after the headers. For an API it is usually JSON.

checker

The script tests/test-runner.mjs. It calls your running server, checks each endpoint against the spec and prints a score.

connection

An open line between your program and the database. You need one before you can send any SQL.

DTO

A shaped copy of an entity for the API: only the fields the client needs, in the format you agreed.

endpoint

One method plus path an API answers, such as GET /api/subjects.

limit

How many records one page holds. The client sends it as limit, and the server caps it.

npm

Node Package Manager: installs libraries such as express into node_modules.

PATH

A Windows setting that lists the folders searched when you type a command. If the MySQL bin folder is not in it, mysql is not recognized.

Placeholders

The ? in a prepared statement. Values fill the ? in order.

port

A number that says which program on the machine should get the message. Web servers use 80 by default.

project template

A saved starting project. A new project made from it already has the folders and the basic code.

request

What the client sends: method, URL, headers and sometimes a body.

route

Matching an HTTP method and a URL path to the function that handles it.

server

A program (or the machine running it) that waits for requests and answers them.

snake_case

Words joined by underscores, like user_name. Common for database columns, not for URIs.

status code

The number that tells the client how it went: 200 OK, 201 Created, 404 Not Found, 500 Server Error.

Summary

Two sizes of DTO: summary has a few fields for lists, detail has everything for one item.

timestamp

A date and time value. A column like last_update can fill itself with the current time.